



Marmara University
Faculty Of Engineering

CSE3038

COMPUTER ORGANIZATION

PROJECT 2

DATE TO : 24 May 2026

Submitted to: Zuhall Öztürk

	Dept	Student Id	Name Surname
1	CSE	150122027	Şeyma Özek
2	CSE	150122060	Fatma Zeynep Kök
3	CSE	150123069	Beyza Çoban
4	CSE	150123081	Sultan Kocagöz

Table of Contents

1. Introduction.....	2
2. Implementation Details.....	2
2.1 lwslt Instruction.....	3
2.2 beqm Instruction.....	3
2.3 swand Instruction.....	4
2.4 swinc Instruction.....	4
2.5 swv Instruction.....	4
2.6 bnpos Instruction.....	5
2.7 balerr Instruction.....	5
3. Datapath Design.....	6
4. Example Runs and Simulation Results.....	6
4.1 lwslt Instruction.....	6
4.2 beqm Instruction.....	7
4.3 swand Instruction.....	9
4.4 swinc Instruction.....	10
4.5 swv Instruction.....	11
4.6 bnpos Instruction.....	13
4.7 balerr Instruction.....	14
5. Conclusion.....	16

1. Introduction

In this project, seven new instructions were added to the given single-cycle MIPS processor. These instructions are lwslt, beqm, swand, swinc, swv, bnpos, and balerr. Some parts of the processor, such as the datapath, control unit, ALU control, memory write logic, and PC selection logic, were updated to support these instructions. The final design keeps the original processor structure and adds the required changes for the new instructions.

2. Implementation Details

Instruction	Type	Opcode / Func	Format	Operation
lwslt	R	func = 20	lwslt \$rd, \$rs, \$rt, shamt	If $\$rt < \text{Mem}[\text{shamt} + \$rs]$, then $\$rd = 1$; otherwise, $\$rd = 0$.
beqm	R	func = 24	beqm \$rd, \$rs, \$rt, shamt	If $\$rt == \text{Mem}[\text{shamt} + \$rs]$, branch to the address stored in \$rd.
swand	R	func = 45	swand \$rd, \$rs, \$rt	Store \$rd into memory address calculated by $\$rt \text{ AND } \rs .
swinc	I	opcode = 44	swinc \$rs, \$rt, imm	Store $\$rt + 1$ into memory address calculated by $\$rs + \text{imm}$.
swv	I	opcode = 39	swv \$rs, \$rt	If $\text{Status}[V] = 1$, store \$rt into memory address \$rs.
bnpos	J	opcode = 24	bnpos Target	If $\text{Status}[Z] = 1$ or $\text{Status}[N] = 1$, branch to the pseudo-direct target address.
balerr	J	opcode = 47	balerr Target	If $\text{Status}[Z] = 1$, $\text{Status}[N] = 1$, and $\text{Status}[V] = 1$, branch to the pseudo-direct target address and store the link address in register \$31.

Table 1. Instruction Descriptions

2.1 lwslt Instruction

For the lwslt instruction, the control unit was extended to detect the R-type funct value of this instruction and to generate a separate lwslt control signal. Since this instruction uses the shamt field in the address calculation, a 5-bit to 32-bit sign extension path was added for the shamt field. This extended value is selected as the second ALU input when the use_shamt signal is active.

The ALU calculates the memory address as $\$rs + shamt$. The data read from this memory address is then compared with $\$rt$ by using the comparator module. In the updated design, the comparator performs signed greater-than and less-than comparisons. Therefore, the less-than condition used by lwslt is evaluated as $signed(\$rt) < signed(Mem[\$rs + shamt])$. The equality output of the comparator is still evaluated with a direct equality comparison.

The result of the signed less-than comparison is combined with the lwslt signal and sent to the write-back path. For this reason, the MemToReg multiplexer was extended so that the comparator result can be written back to the register file. In the control unit, regwrite is enabled for this instruction, and the destination register is selected from the R-type destination field.

2.2 beqm Instruction

For the beqm instruction, a separate beqm signal was added to the control unit by decoding its R-type funct value. Similar to lwslt, this instruction also uses the shamt field for memory address calculation. Therefore, the same 5-bit sign extension path and use_shamt selection logic are used.

The ALU calculates the memory address as $\$rs + shamt$. The memory output and the $\$rt$ value are connected to the comparator module. Although the comparator performs signed greater-than and less-than comparisons, the equality output used by beqm is evaluated with a direct equality comparison. Therefore, the beqm condition is true when $\$rt == Mem[\$rs + shamt]$.

When the equality condition is true, the ifBeqm signal becomes active. The PC selection path then selects the value read from the third register output as the next PC. This third register output corresponds to the register specified by the rd field. For this instruction, register write is disabled because the instruction only changes the program counter.

2.3 swand Instruction

For the swand instruction, the control unit was updated to detect its R-type funct value and generate the swand control signal. The ALU control unit was also updated so that the ALU performs the required logical AND operation for this instruction.

The ALU calculates the memory address by applying the bitwise AND operation between \$rs and \$rt. The calculated ALU result is selected as the data memory address through the memory address path. The memory write path was also modified because this instruction does not use the normal store write-data source. A new multiplexer was added before the data memory write input. This multiplexer allows the third register output, which corresponds to \$rd, to be selected as the value written to memory when the swand signal is active.

In the control unit, memwrite is enabled for this instruction, while regwrite is disabled. Therefore, swand writes the value of \$rd into the memory address calculated by \$rs AND \$rt.

2.4 swinc Instruction

For the swinc instruction, the control unit was extended to decode its opcode and generate the swinc signal. The normal sign-extended immediate path is used for address calculation, and the ALU calculates the memory address as $\$rs + \text{immediate}$ through the existing ALU source selection logic.

An additional adder was added to the datapath to produce the incremented register value. This adder takes the second register output, which corresponds to \$rt, and adds 1 to it. Its output is connected to the memory write-data multiplexer. When the swinc signal is active, this incremented value is selected as the data memory write input.

In the control unit, memwrite is enabled and regwrite is disabled for this instruction. Therefore, swinc stores $\$rt + 1$ into the memory address calculated by $\$rs + \text{immediate}$.

2.5 swv Instruction

For the swv instruction, the control unit was updated to decode its opcode and generate the swv control signal. This instruction uses the overflow bit stored in the status register. The status register stores the ALU-generated V, N, and Z flags, and it is updated after every instruction on the positive clock edge. Therefore, swv checks the current stored value of Status[V] during execution.

A new multiplexer was added before the data memory address input. In the normal case, the data memory address comes from the ALU result. However, for the swv instruction, the memory address must be taken directly from \$rs. Therefore, this address multiplexer selects Read Data 1 as the memory address when the swv signal is active.

The memory write control was also modified. The control unit generates the normal memwrite signal, but the actual memory write signal is enabled only when the memwrite signal is active and instruction is not swv or when Status[V] is equal to 1. This allows normal store instructions to work unchanged while adding conditional memory write support for swv.

As a result, swv stores \$rt into the memory address stored in \$rs only if Status[V] is 1.

2.6 bnpos Instruction

For the bnpos instruction, the control unit was extended to decode its J-type opcode and generate the bnpos signal. The pseudo-direct target address path was added by shifting the 26-bit target field left by two bits and combining it with the upper bits of PC + 4.

This instruction uses the stored status register outputs. Since the status register is updated after every instruction, bnpos checks the current stored Z and N flags at the time of execution. The zero and negative bits are combined with an OR gate. If Status[Z] = 1 or Status[N] = 1, the do_bnpos_branch signal becomes active.

The final PC multiplexer then selects the pseudo-direct target address as the next PC. Therefore, bnpos performs a conditional pseudo-direct branch based on the stored Z and N status bits.

2.7 balerr Instruction

For the balerr instruction, the control unit was updated to decode its J-type opcode and generate the balerr signal. Since this instruction also performs a link operation, the register destination selection logic was extended. A new path was added to select register \$31 as the destination register when balerr is active.

The write-back multiplexer was also extended to include PC + 4 as a possible write-back value. This allows the processor to write the link address into register \$31.

The branch condition of this instruction is based on the stored status register values. Since the status register is updated after every instruction, balerr checks the current Z, N, and V flags. These three bits are combined with AND logic. If Status[Z] = 1, Status[N] = 1, and Status[V] = 1, the do_balerr_branch signal becomes active.

In this case, the final PC multiplexer selects the pseudo-direct target address as the next PC, and PC + 4 is written into register \$31. Therefore, balrr works as a conditional branch-and-link instruction.

3. Datapath Design

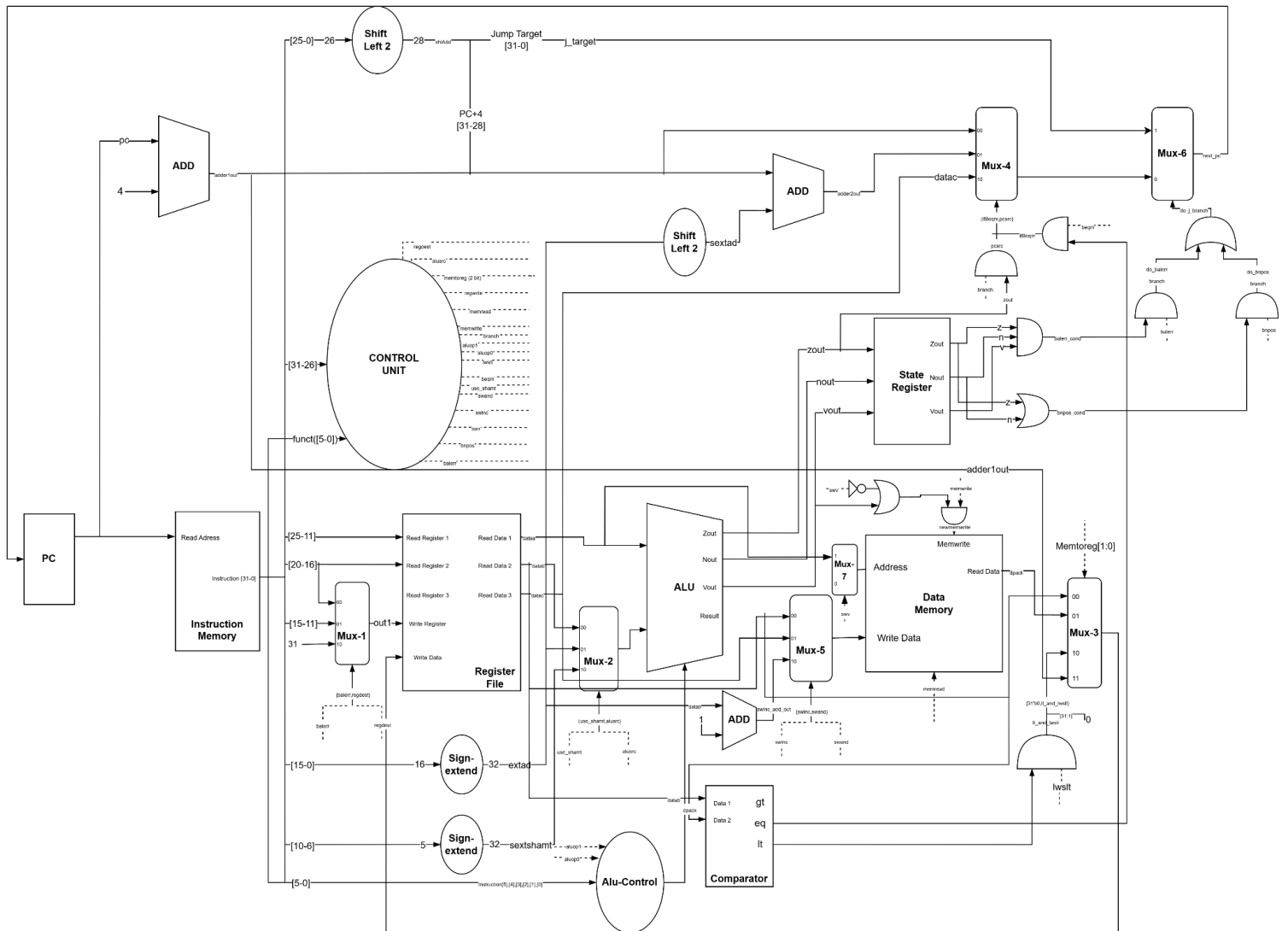


Figure 1. Datapath design used for the implemented instructions.

4. Example Runs and Simulation Results

4.1 lwslt Instruction

`lwslt $3, $1, $2, 4`

If $R2 == \text{Mem}[R1 + 4]$, $PC = R5$.

Initial Values:

$R1 = 8$

$R2 = 9$

$R5 = 0x20$

$\text{Mem}[12] = 9$

Expected Result:

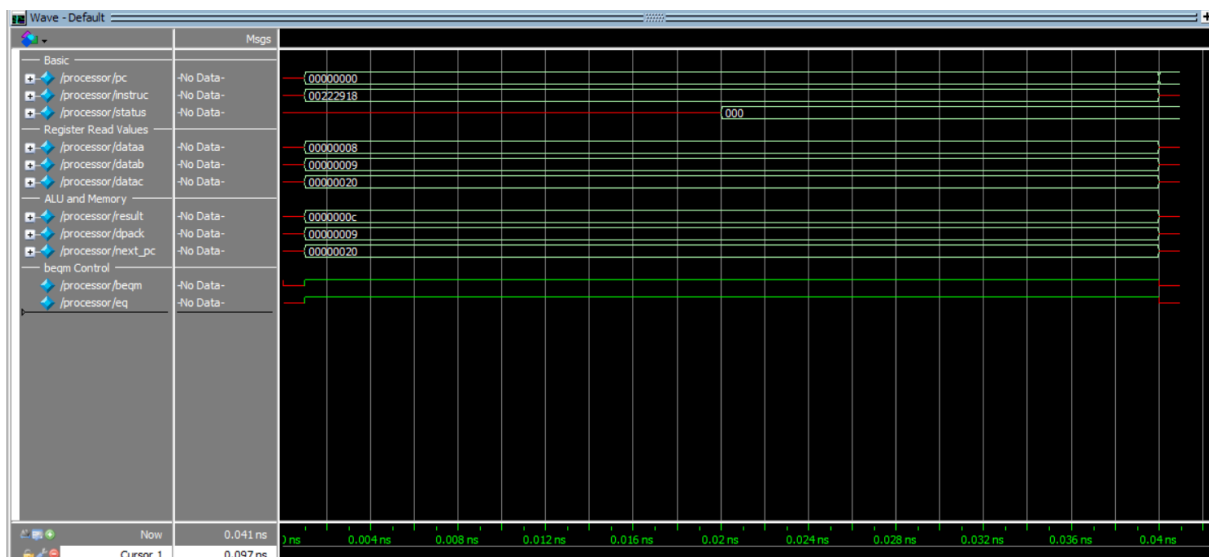
Since $R2$ is equal to $\text{Mem}[12]$, $PC = 00000020$

Simulation Result:

```
# ----- BEQM BEFORE BRANCH -----
VSIM 115> examine -radix hex sim:/processor/pc
# 00000000
VSIM 116> examine -radix hex sim:/processor/instruc
# 00222918
VSIM 117> examine -radix binary sim:/processor/beqm
# 1
VSIM 118> examine -radix binary sim:/processor/eq
# 1
VSIM 119> examine -radix hex sim:/processor/result
# 0000000c
VSIM 120> examine -radix hex sim:/processor/dpack
# 00000009
VSIM 121> examine -radix hex sim:/processor/next_pc
# 00000020

VSIM 122> run 21 ps
#          20 PC 00000000  INST 00222918  STAT=000  do_j=0  MEM[8]=xxxxxxx  MEM[12]=00000009
#          40 PC 00000020  INST xxxxxxxx  STAT=000  do_j=0  MEM[8]=xxxxxxx  MEM[12]=00000009

VSIM 123> echo "----- BEQM AFTER BRANCH -----"
# ----- BEQM AFTER BRANCH -----
VSIM 124> examine -radix hex sim:/processor/pc
# 00000020
```



For the beqm instruction, $R1$ was initialized to 8, $R2$ was initialized to 9, and $R5$ was initialized to $0x20$. The ALU calculated the memory address as $R1 + \text{shamt} = 12$. Since the memory value at address 12 was 9 and $R2$ was also 9, the equality condition became true. Therefore, the next PC was selected as $0x20$, which is the value stored in $R5$.

4.3 swand Instruction

swand \$4, \$1, \$2

Mem[R2 AND R1] = R4

Initial Values:

R1 = 12

R2 = 10

R4 = AABBCDD

Mem[8] = 00000000

12 AND 10 = 8

Expected Result:

Mem[8] = AABBCDD

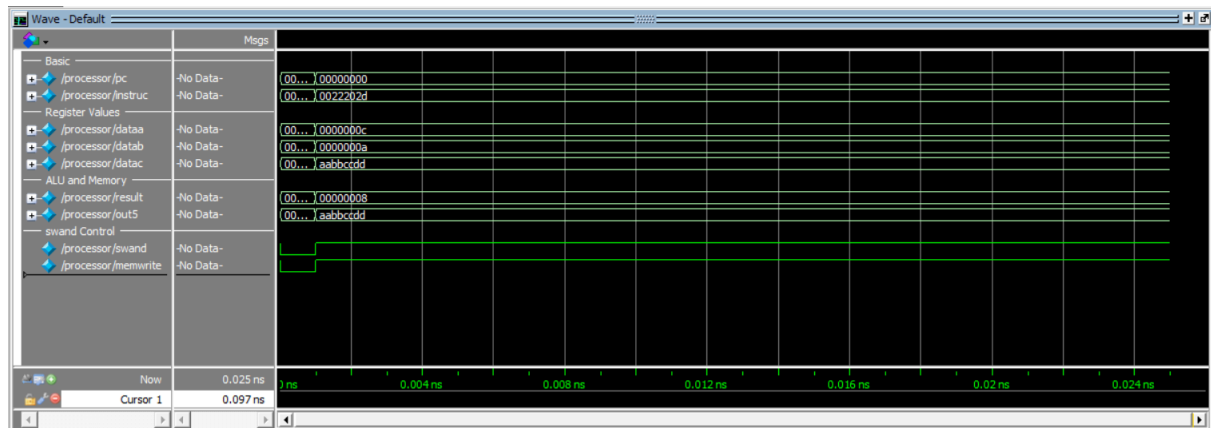
Simulation Result:

```
VSIM 164> run 9 ps
# 1 PC 00000000 INST 0022202d STAT=xxx do_j=0 MEM[8]=00000000 MEM[12]=xxxxxxx

VSIM 165> echo "----- SWAND BEFORE MEMORY WRITE -----"
# ----- SWAND BEFORE MEMORY WRITE -----
VSIM 166> examine -radix hex sim:/processor/pc
# 00000000
VSIM 167> examine -radix hex sim:/processor/instruc
# 0022202d
VSIM 168> examine -radix binary sim:/processor/swand
# 1
VSIM 169> examine -radix hex sim:/processor/dataaa
# 0000000c
VSIM 170> examine -radix hex sim:/processor/datab
# 0000000a
VSIM 171> examine -radix hex sim:/processor/datac
# aabbccdd
VSIM 172> examine -radix hex sim:/processor/result
# 00000008
VSIM 173> examine -radix hex sim:/processor/out5
# aabbccdd
VSIM 174> examine -radix binary sim:/processor/memwrite
# 1

VSIM 175> run 15 ps
# 20 PC 00000000 INST 0022202d STAT=000 do_j=0 MEM[8]=aabbccdd MEM[12]=xxxxxxx

VSIM 176> echo "----- SWAND AFTER MEMORY WRITE -----"
# ----- SWAND AFTER MEMORY WRITE -----
VSIM 177> examine -radix hex {sim:/processor/datmem[8]}
# aa
VSIM 178> examine -radix hex {sim:/processor/datmem[9]}
# bb
VSIM 179> examine -radix hex {sim:/processor/datmem[10]}
# cc
VSIM 180> examine -radix hex {sim:/processor/datmem[11]}
# dd
```



For the swand instruction, register R1 was initialized to 12, register R2 was initialized to 10, and register R4 was initialized to AABBCDD. The instruction calculates the memory address by applying the bitwise AND operation between R2 and R1. Since 12 AND 10 equals 8, the target memory address is 8. Before execution, Mem[8] was initialized to 00000000. After the instruction was executed, the value stored in R4 was written to memory address 8. Therefore, the expected result is Mem[8] = AABBCDD.

4.4 swinc Instruction

swinc \$1, \$2, 4

Mem[R1 + 4] = R2 + 1

Initial Values:

R1 = 8

R2 = 10

Expected Result:

Address = R1 + 4 = 12

Stored value = R2 + 1 = 11

Mem[12] = 0000000B

Simulation Result:

```

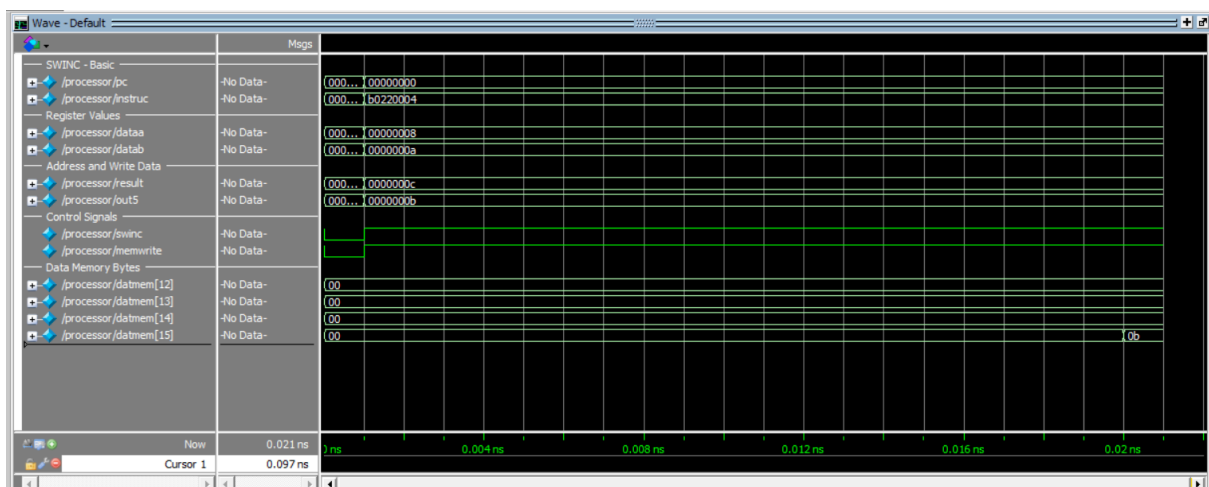
VSIM 185> run 18 ps
# 1 PC 00000000 INST b0220004 STAT=xxx do_j=0 MEM[8]=xxxxxxxx MEM[12]=00000000

VSIM 186> echo "----- SWINC BEFORE MEMORY WRITE -----"
# ----- SWINC BEFORE MEMORY WRITE -----
VSIM 187> examine -radix hex sim:/processor/pc
# 00000000
VSIM 188> examine -radix hex sim:/processor/instruc
# b0220004
VSIM 189> examine -radix binary sim:/processor/swinc
# 1
VSIM 190> examine -radix hex sim:/processor/dataa
# 00000008
VSIM 191> examine -radix hex sim:/processor/datab
# 0000000a
VSIM 192> examine -radix hex sim:/processor/result
# 0000000c
VSIM 193> examine -radix hex sim:/processor/out5
# 0000000b
VSIM 194> examine -radix binary sim:/processor/memwrite
# 1

VSIM 195> run 2 ps
# 20 PC 00000000 INST b0220004 STAT=xxx do_j=0 MEM[8]=xxxxxxxx MEM[12]=0000000b

VSIM 196> echo "----- SWINC AFTER MEMORY WRITE -----"
# ----- SWINC AFTER MEMORY WRITE -----
VSIM 197> examine -radix hex {sim:/processor/datmem[12]}
# 00
VSIM 198> examine -radix hex {sim:/processor/datmem[13]}
# 00
VSIM 199> examine -radix hex {sim:/processor/datmem[14]}
# 00
VSIM 200> examine -radix hex {sim:/processor/datmem[15]}
# 0b

```



For the swinc instruction, R1 was initialized to 8 and R2 was initialized to 10. The processor calculated the memory address as $R1 + \text{immediate} = 8 + 4 = 12$. The value written to memory was $R2 + 1$, which is 11 or 0000000B in hexadecimal. Since the data memory is byte-addressable, this value was stored as 00 00 00 0B in memory addresses 12–15. The simulation result confirms that the swinc instruction worked correctly.

4.5 swv Instruction

swv \$5, \$4

If Status[V] = 1:

Mem[R5] = R4

Initial Values:

R5 = 00000008

R4 = AABBCDD

Status[V] = 1

Mem[8] = 00000000

Expected Result:

Mem[8] = AABBCDD

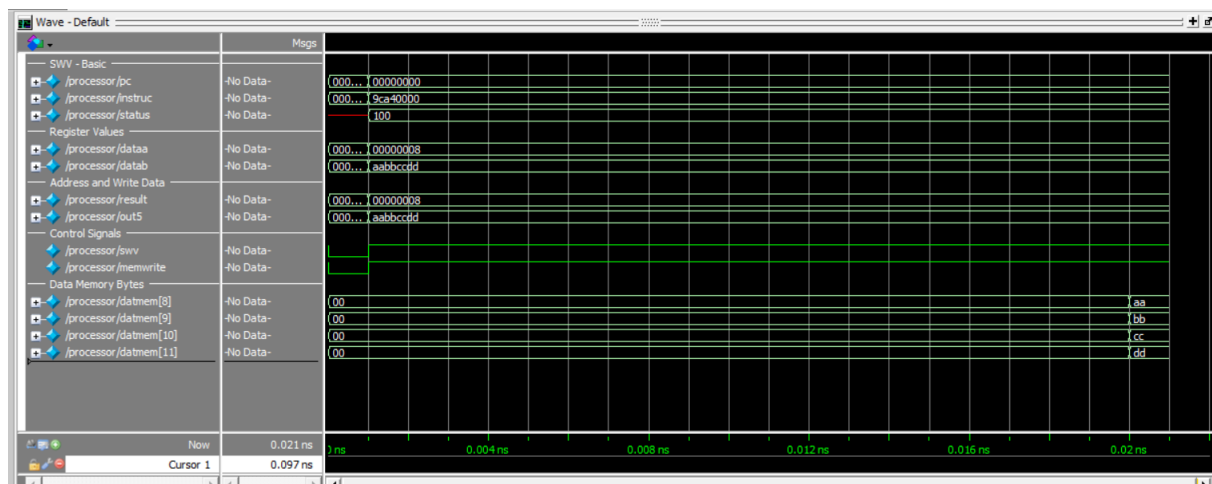
Simulation Result:

```
VSIM 315> run 18 ps
# 1 PC 00000000 INST 9ca40000 STAT=100 do_j=0 MEM[8]=00000000 MEM[12]=xxxxxxx

VSIM 316> echo "----- SWV BEFORE MEMORY WRITE -----"
# ----- SWV BEFORE MEMORY WRITE -----
VSIM 317> examine -radix hex sim:/processor/pc
# 00000000
VSIM 318> examine -radix hex sim:/processor/instruc
# 9ca40000
VSIM 319> examine -radix binary sim:/processor/status
# 100
VSIM 320> examine -radix binary sim:/processor/swv
# 1
VSIM 321> examine -radix hex sim:/processor/dataa
# 00000008
VSIM 322> examine -radix hex sim:/processor/datab
# aabbccdd
VSIM 323> examine -radix hex sim:/processor/result
# 00000008
VSIM 324> examine -radix hex sim:/processor/out5
# aabbccdd
VSIM 325> examine -radix binary sim:/processor/memwrite
# 1

VSIM 326> run 2 ps
# 20 PC 00000000 INST 9ca40000 STAT=100 do_j=0 MEM[8]=aabbccdd MEM[12]=xxxxxxx

VSIM 327> echo "----- SWV AFTER MEMORY WRITE -----"
# ----- SWV AFTER MEMORY WRITE -----
VSIM 328> examine -radix hex {sim:/processor/datmem[8]}
# aa
VSIM 329> examine -radix hex {sim:/processor/datmem[9]}
# bb
VSIM 330> examine -radix hex {sim:/processor/datmem[10]}
# cc
VSIM 331> examine -radix hex {sim:/processor/datmem[11]}
# dd
```



For the swv instruction, register R5 was initialized to 8 and register R4 was initialized to AABBCDD. The overflow flag V in the status register was set to 1. Since the swv instruction writes to memory only when Status[V] is equal to 1, the memory write operation was enabled. The processor used R5 as the memory address and wrote the value of R4 to memory. The simulation result shows that the memory bytes at addresses 8–11 were updated to AA BB CC DD, which represents the 32-bit value AABBCDD.

4.6 bnpos Instruction

bnpos Target

If Status[Z] = 1 OR Status[N] = 1:

PC = pseudo-direct target address

Initial Values:

Status[Z] = 1

Status[N] = 0

Target = 2

PC = 00000000

Expected Result:

Target address = Target << 2

Target address = 2 << 2 = 8

PC = 00000008

Simulation Result:

```

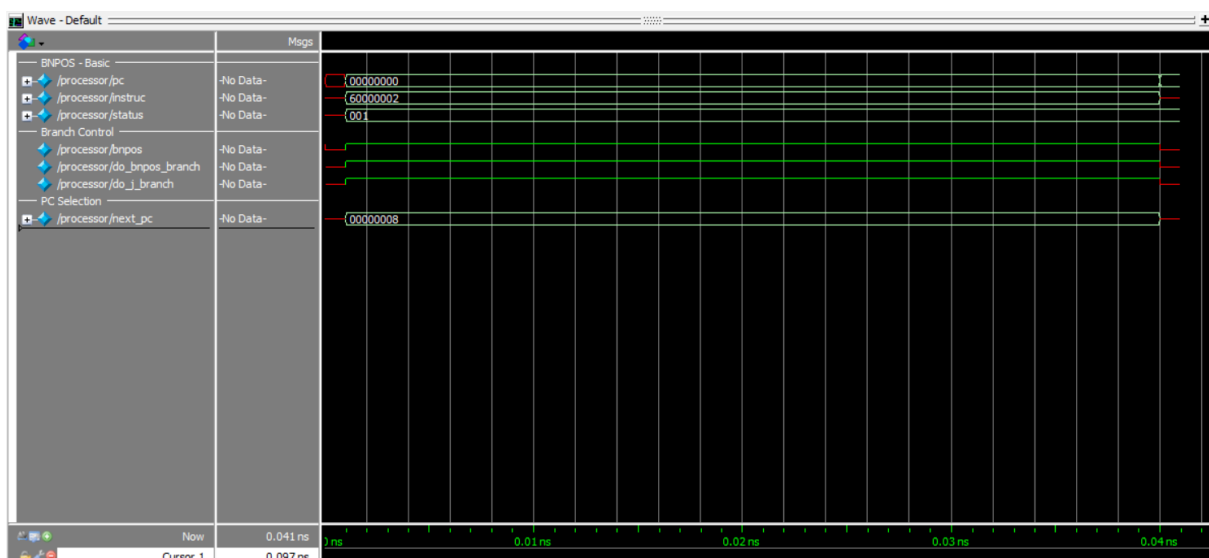
VSIM 363> force -deposit sim:/processor/pc 32'h00000000
VSIM 364> force -freeze sim:/processor/status 3'b001 0
VSIM 365> run 18 ps
#          1 PC 00000000  INST 60000002  STAT=001  do_j=1  MEM[8]=xxxxxxx  MEM[12]=xxxxxxx

VSIM 366> echo "----- BNPOS BEFORE BRANCH -----"
# ----- BNPOS BEFORE BRANCH -----
VSIM 367> examine -radix hex sim:/processor/pc
# 00000000
VSIM 368> examine -radix hex sim:/processor/instruc
# 60000002
VSIM 369> examine -radix binary sim:/processor/status
# 001
VSIM 370> examine -radix binary sim:/processor/bnpos
# 1
VSIM 371> examine -radix binary sim:/processor/do_bnpos_branch
# 1
VSIM 372> examine -radix binary sim:/processor/do_j_branch
# 1
VSIM 373> examine -radix hex sim:/processor/next_pc
# 00000008

VSIM 374> run 22 ps
#          40 PC 00000008  INST xxxxxxxx  STAT=001  do_j=x  MEM[8]=xxxxxxx  MEM[12]=xxxxxxx

VSIM 375> echo "----- BNPOS AFTER BRANCH -----"
# ----- BNPOS AFTER BRANCH -----
VSIM 376> examine -radix hex sim:/processor/pc
# 00000008

```



For the bnpos instruction, the zero flag was manually set to 1 in the status register. The branch condition of this instruction is satisfied when either Z or N is equal to 1. Since Status[Z] was set to 1, the processor selected the pseudo-direct target address as the next PC. The target field was 2, so the target address was calculated as $2 \ll 2 = 8$. The simulation result shows that the PC was updated to 00000008.

4.7 balerr Instruction

balerr Target

If Status[Z] = 1 AND Status[N] = 1 AND Status[V] = 1:

PC = pseudo-direct target address

R31 = PC + 4

Initial Values:

Status[Z] = 1

Status[N] = 1

Status[V] = 1

Target = 2

PC = 00000000

R31 = 00000000

Expected Result:

Target address = Target << 2

Target address = 2 << 2 = 8

PC = 00000008

R31 = 00000004

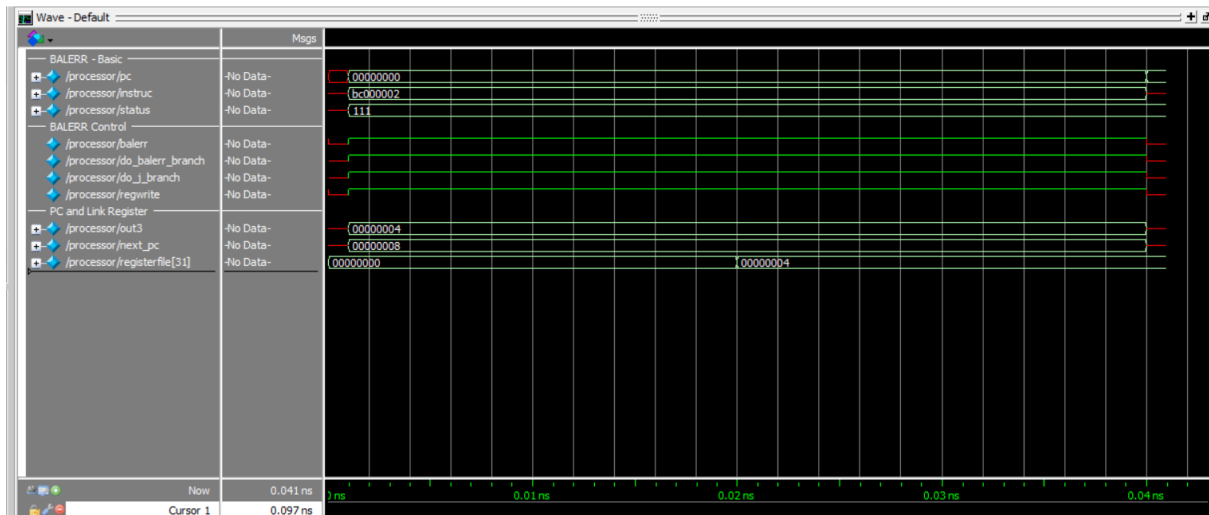
Simulation Result:

```
VSIM 395> force -freeze sim:/processor/status 3'b111 0
VSIM 396> run 18 ps
# 1 PC 00000000 INST bc000002 STAT=111 do_j=1 MEM[8]=xxxxxxxx MEM[12]=xxxxxxxx

VSIM 397> echo "----- BALERR BEFORE BRANCH -----"
# ----- BALERR BEFORE BRANCH -----
VSIM 398> examine -radix hex sim:/processor/pc
# 00000000
VSIM 399> examine -radix hex sim:/processor/instruc
# bc000002
VSIM 400> examine -radix binary sim:/processor/status
# 111
VSIM 401> examine -radix binary sim:/processor/balerr
# 1
VSIM 402> examine -radix binary sim:/processor/do_balerr_branch
# 1
VSIM 403> examine -radix binary sim:/processor/do_j_branch
# 1
VSIM 404> examine -radix binary sim:/processor/regwrite
# 1
VSIM 405> examine -radix hex sim:/processor/out3
# 00000004
VSIM 406> examine -radix hex sim:/processor/next_pc
# 00000008

VSIM 407> run 22 ps
# 40 PC 00000008 INST xxxxxxxx STAT=111 do_j=x MEM[8]=xxxxxxxx MEM[12]=xxxxxxxx

VSIM 408> echo "----- BALERR AFTER BRANCH -----"
# ----- BALERR AFTER BRANCH -----
VSIM 409> examine -radix hex sim:/processor/pc
# 00000008
VSIM 410> examine -radix hex {sim:/processor/registerfile[31]}
# 00000004
```



For the balerr instruction, the status register was manually set to 111. This means that the Z, N, and V flags were all set to 1. Since the branch condition of balerr was satisfied, the processor selected the pseudo-direct target address as the next PC. The target field was 2, so the target address was calculated as $2 \ll 2 = 8$. Since balerr also works as a branch-and-link instruction, PC + 4 was written into register R31. The simulation result shows that the PC was updated to 00000008 and R31 was updated to 00000004.

5. Conclusion

The implementation was completed by extending the existing single-cycle MIPS processor with the required datapath and control unit changes. The added instructions were supported by using additional control signals, multiplexers, comparator logic, a continuously updated status register, conditional memory write logic, memory write-data selection logic, and updated PC selection logic.

The status register was designed to store the ALU-generated V, N, and Z flags after every instruction. These stored flags were then used by the status-dependent instructions. The comparator was also updated to perform signed greater-than and less-than comparisons, which allowed the lwslt instruction to evaluate signed values correctly.

The data memory address path was modified by adding a multiplexer before the memory address input. This allowed normal memory instructions to use the ALU result as the address while allowing the swv instruction to use \$rs directly as the memory address. In addition, the memory write-data path was extended so that swand could write \$rd and swinc could write \$rt + 1 to memory.

All instructions were tested separately in ModelSim with different initialization files for instruction memory, register file, and data memory. The simulation results showed that the expected outputs were obtained for each case. Memory-related instructions updated the correct memory locations, comparison-based instructions produced the correct result, and branch instructions updated the PC when their conditions were satisfied.

Based on the simulation results, the final design correctly supports all seven assigned instructions in the single-cycle datapath.